Local AI Image Generation

Complete Workflow Guide

ComfyUI + FLUX.2 Klein + n8n + Ollama Qwen

Text → Enhanced Prompt → AI Image → Email Delivery

Tested on Windows 11 • n8n in Docker • May 2026

What This Guide Covers

ComfyUI Workflow	How FLUX.2 Klein 4B generates images locally on your GPU
n8n Automation	Full 9-node workflow that takes a chat message and emails back a generated image
Ollama Qwen	Local LLM that turns your simple description into a rich image prompt
Docker Tips	Key differences when running n8n inside Docker vs. natively
Troubleshooting	Common errors and exactly how to fix them

■ How The Full System Works

Before diving into setup, here's the big picture. Three separate applications run on your Windows machine simultaneously, each with a specific job:

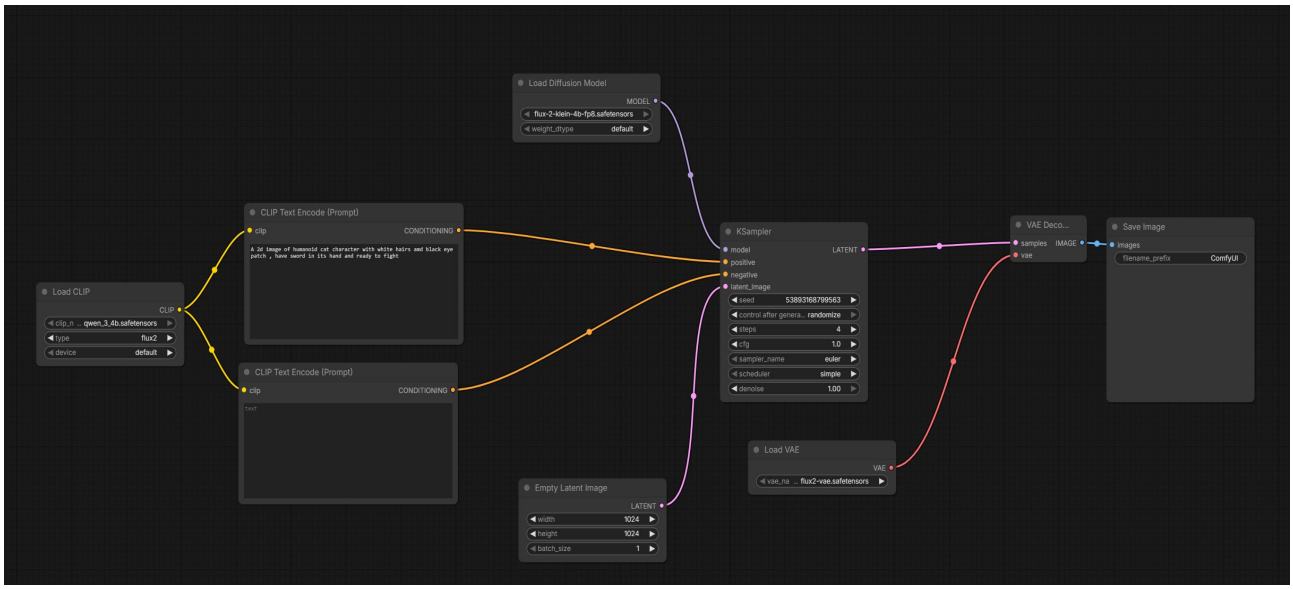
■ ComfyUI	The image factory. Loads FLUX.2 Klein into your GPU and generates images. Must be running in the background (run_nvidia_gpu.bat open) the entire time.
■ Ollama	Runs Qwen 3 locally. Transforms your simple text description into a rich, detailed prompt before it reaches the image model.
■ n8n	The automation brain. Receives your chat message, calls Ollama, calls ComfyUI, waits for the image, downloads it, and emails it — all automatically.

The Request Flow

1	**You type**	a simple image idea in the n8n chat
2	**Qwen (Ollama)**	rewrites it into a detailed, vivid image prompt
3	**n8n → ComfyUI**	sends the full workflow JSON to ComfyUI's API at port 8188
4	**GPU computes**	FLUX.2 Klein generates the image (takes 10–60 seconds)
5	**n8n polls**	checks every 8 seconds "is the image ready yet?"
6	**n8n downloads**	fetches the finished image binary from ComfyUI
7	**Gmail sends**	emails you the image as an attachment

■ ComfyUI Workflow

ComfyUI uses a node-based visual interface where each node performs one specific task. Your workflow loads three model files, encodes the text prompt, generates a latent image on the GPU, decodes it to pixels, and saves the PNG file.



Your ComfyUI workflow — all green nodes connected and working

Node Breakdown

1 **Load Diffusion Model**

UNETLoader

Loads the main FLUX.2 Klein FP8 model into GPU memory. File: flux-2-klein-4b-fp8.safetensors

2 **Load CLIP**

CLIPLoader

Loads the Qwen 3 4B text encoder that understands your prompts. File: qwen_3_4b.safetensors | Type: flux2

3 **Load VAE**

VAELoader

Loads the Variational Autoencoder that converts latent data to pixels. File: flux2-vae.safetensors ← must use FLUX.2 VAE, not FLUX.1

4 **CLIP Text Encode (Positive)**

CLIPTextEncode

Your image prompt goes here. n8n injects the Qwen-enhanced prompt into this node dynamically via the API.

5 **CLIP Text Encode (Negative)**

CLIPTextEncode

Negative prompt — left empty for FLUX.2 Klein. This model works best without negative prompts.

6 **Empty Latent Image**

EmptyLatentImage

Sets output resolution. Currently 1024×1024 px. You can change width/height here.

7 **KSampler**

KSampler

The core generation node. Key settings: Sampler: euler | Scheduler: simple | Steps: 4 | CFG: 1.0

8 **VAE Decode**

VAEDecode

Converts the generated latent tensor into a viewable PNG image.

9 **Save Image**

SaveImage

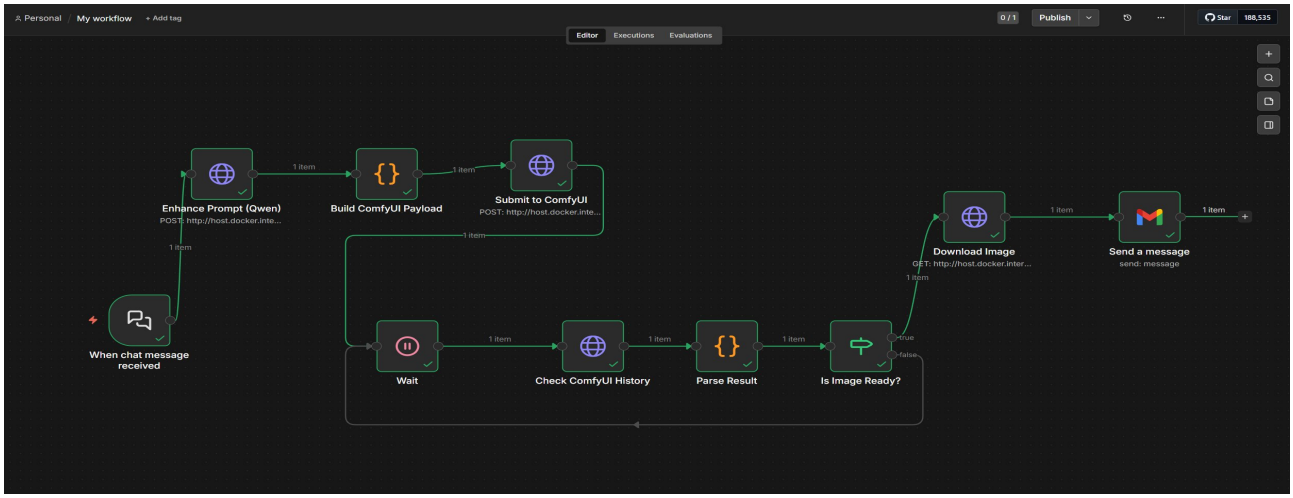
Saves the output PNG to ComfyUI's output folder. Prefix: ComfyUI → files saved as ComfyUI_00001_.png etc.

■ **Warning:** FLUX.2 Klein uses exactly 4 steps with CFG 1.0. Do NOT increase steps thinking it will improve quality — it will break the image. This model is specifically trained for 4-step generation.

■ **Tip:** Write prompts as flowing prose, not keyword lists. "A warrior standing on a cliff at sunset with dramatic lighting" works much better than "warrior, cliff, sunset, dramatic, 8k, masterpiece".

■ n8n Workflow — All 9 Nodes

The n8n workflow is the automation layer. It orchestrates all the moving parts — calling Ollama, submitting to ComfyUI, waiting for the result, and delivering the image via email.



Your completed n8n workflow — all nodes connected with the polling loop visible

■ **Warning:** DOCKER USERS — Critical: n8n runs inside Docker and cannot reach your Windows machine at 127.0.0.1. Use host.docker.internal instead of localhost or 127.0.0.1 in every URL. Example: http://host.docker.internal:8188/prompt

NODE 1 — When Chat Message Received

The entry point. Creates a chat interface in n8n where you type your image idea.

Setting	Value
Node Type	`Chat Trigger`
All settings	`Leave as default`
Output field	`{{ $json.chatInput }}`

NODE 2 — Enhance Prompt (Qwen)

Sends your simple idea to Ollama's local Qwen model which rewrites it as a detailed, vivid image generation prompt before it reaches FLUX.

Setting	Value
Node Type	`HTTP Request`
Method	`POST`
URL (native)	`http://127.0.0.1:11434/api/chat`

URL (Docker)	`http://host.docker.internal:11434/api/chat`
Body Type	`JSON`

■ **Info:** Check your exact Qwen model name with ollama list in terminal. Update "qwen3:9b" in the body JSON if your model name differs.

NODE 3 — Build ComfyUI Payload

A Code node that builds the ComfyUI workflow JSON programmatically. This is necessary because the seed must be a proper JavaScript number — not a string — or ComfyUI will reject the workflow silently.

Setting	Value
Node Type	`Code`
Language	`JavaScript`
Mode	`Run Once for All Items`
Key job	`Injects enhanced prompt into node "5" + generates random seed`

■ **Info:** This is why we use a Code node instead of putting the JSON directly in the HTTP Request body. Expressions like ={{ Math.floor(...) }} inside a JSON body field produce a string, but ComfyUI requires seed to be a number type. The Code node guarantees correct types.

NODE 4 — Submit to ComfyUI

POSTs the workflow JSON to ComfyUI's API. ComfyUI queues the job and returns a unique prompt_id that we use to track when the image is done.

Setting	Value
Node Type	`HTTP Request`
Method	`POST ← must be POST not GET`
URL (native)	`http://127.0.0.1:8188/prompt`
URL (Docker)	`http://host.docker.internal:8188/prompt`
Body Type	`JSON`
Body value	`={{ JSON.stringify($json) }}`
Expected response	`{ "prompt_id": "abc-123...", "number": 1 }`

■ **Warning:** If this node returns only { "exec_info": { "queue_remaining": 0 } } with no prompt_id, the workflow JSON had a type error. Make sure you have the Code node before this one.

NODE 5 — Wait 8 Seconds

Pauses execution to give the GPU time to generate the image.

Setting	Value
Node Type	`Wait`
Resume	`After time interval`
Amount	`8 seconds (increase to 20 if GPU is slow)`

NODE 6 — Check ComfyUI History

Calls ComfyUI's history endpoint to check if the image generation job is complete.

Setting	Value
Node Type	`HTTP Request`
Method	`GET`
URL (Docker)	`http://host.docker.internal:8188/history/{{ $('Submit to ComfyUI').first().json.prompt_id }}`

NODE 7 — Parse Result

A Code node that reads the history response and extracts the filename if generation is complete. Returns completed: true/false so the next node can decide what to do.

Setting	Value
Node Type	`Code`
Returns if done	`{ completed: true, filename: "ComfyUI_00001_.png", ... }`
Returns if not done	`{ completed: false, promptId: "..." }`

NODE 8 — Is Image Ready? (If node)

Routes the execution based on whether the image is done. The false path loops back to the Wait node creating a polling cycle.

Setting	Value
Node Type	`If`
Condition	`Boolean`
Value	`{{ $json.completed }}`
Operation	`is true`
true output	`→ Download Image node`
false output	`→ loops back to Wait node`

■ **Tip:** The false → Wait loop is what makes this workflow smart. Instead of guessing how long generation takes, n8n keeps checking every 8 seconds until ComfyUI confirms the image is ready, then automatically continues.

NODE 9 — Download Image

Downloads the finished image as binary data from ComfyUI's view endpoint.

Setting	Value
Node Type	`HTTP Request`
Method	`GET`
URL (Docker)	`http://host.docker.internal:8188/view`
Query param: filename	`{{ $json.filename }}`
Query param: subfolder	`{{ $json.subfolder }}`
Query param: type	`output`
Response Format	`File`
Output Binary Field	`image`

NODE 10 — Send via Gmail

Emails the generated image as an attachment. Requires Gmail OAuth2 credentials.

Setting	Value
Node Type	`Gmail`
Resource	`Message`
Operation	`Send`

Attachment field	`image (binary field from Download node)`

■ **Info:** To set up Gmail: Google Cloud Console → Create Project → Enable Gmail API → Create OAuth2 credentials → paste Client ID and Secret into n8n credentials.

■ Running n8n in Docker — Key Differences

When n8n runs inside a Docker container, it is isolated from your Windows machine. This means standard localhost addresses do not work — Docker cannot see outside its own network using 127.0.0.1 because that address points back inside the container itself.

Service	Native / Non-Docker	n8n in Docker
Ollama	http://127.0.0.1:11434	http://host.docker.internal:11434
ComfyUI /prompt	http://127.0.0.1:8188/prompt	http://host.docker.internal:8188/prompt
ComfyUI /history	http://127.0.0.1:8188/history/...	http://host.docker.internal:8188/history/...
ComfyUI /view	http://127.0.0.1:8188/view	http://host.docker.internal:8188/view

■ **Info:** host.docker.internal is Docker Desktop's built-in DNS name that always points to the Windows host machine running Docker. It works on both Windows and Mac with Docker Desktop.

■ Troubleshooting — Common Errors & Fixes

■ Connection refused / service offline

Cause: Ollama or ComfyUI is not running.

Fix:
```
Start ComfyUI: run_nvidia_gpu.bat
Start Ollama: ollama serve in terminal
```

■ exec_info returned but no prompt_id

Cause: ComfyUI received the request but rejected the workflow JSON.

Fix:
```
Make sure the Build ComfyUI Payload Code node is before Submit to ComfyUI.
The seed must be a number type, not a string.
```

■ History check returns 404

Cause: The prompt_id expression is empty — previous node data not available.

Fix:
```
Run the full workflow from Node 1. Do not test Node 6 (Check History) in isolation.
```

■ Image is black or heavily distorted

Cause: Wrong VAE file being used.

Fix:
```
Use flux2-vae.safetensors only. Do not use FLUX.1 ae.safetensors or any SDXL VAE.
```

■ Polling loop runs forever and never completes

Cause: GPU is slow or Wait time is too short.

Fix:
```
Increase Wait node from 8 seconds to 20 or 30 seconds.
```

■ Gmail credentials error

Cause: OAuth2 not configured.

Fix:
```
Google Cloud Console → Enable Gmail API → Create OAuth2 credentials → Add to n8n credentials.
```

■ Write binary file not writable

Cause:	n8n in Docker cannot write to Windows C:\ paths.
Fix:	`Use /tmp/filename.png to write inside the container, or skip the write node and use Gmail directly.`

■ Quick Reference

Model Files

Setting	Value
Diffusion Model	`flux-2-klein-4b-fp8.safetensors → models/unet/`
Text Encoder	`qwen_3_4b.safetensors → models/clip/`
VAE	`flux2-vae.safetensors → models/vae/`

Ports & Services

Setting	Value
ComfyUI API	`http://host.docker.internal:8188`
Ollama API	`http://host.docker.internal:11434`
n8n Interface	`http://localhost:5678 (in browser)`

KSampler Settings

Setting	Value
Sampler	`euler (NOT euler_ancestral)`
Scheduler	`simple`
Steps	`4 (do not change)`
CFG Scale	`1.0 (do not change)`
Resolution	`1024 × 1024 (default)`

Before Every Run — Checklist

- ■ ComfyUI is running (run_nvidia_gpu.bat terminal is open)
- ■ Ollama is running (ollama serve or ollama run qwen3:9b)
- ■ n8n workflow is Active (toggle in top right of n8n)
- ■ Gmail credentials connected in n8n